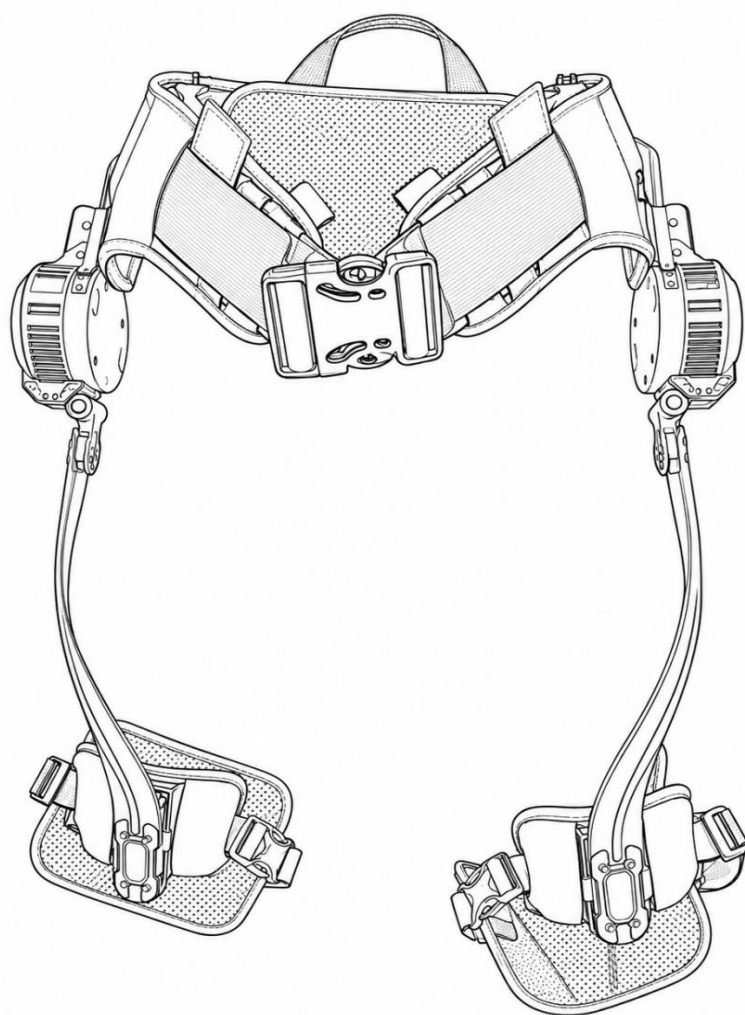


林-I 系列髋关节外骨骼 二次开发

林-I 系列髋关节外骨骼

二次开发

V1.0 2026/08



文档更新说明

版本	版本更新说明	负责人	审核人	发布日期
V1.0	初稿	驭介智能嵌入式团队	驭介智能嵌入式团队	2026.8.15

免责声明

本产品及配套软硬件资料主要用于科教、科研与技术交流。二次开发可能改变设备原有的控制逻辑、运动性能和安全保护机制，并可能引发设备失控、机械碰撞、部件损坏或人身伤害等风险。开发者在进行源代码修改、硬件改装、通信接入、控制参数调整及算法替换前，应充分理解相关技术原理与潜在风险，并具备必要的开发和安全测试能力。

二次开发完成后，开发者应首先在断电、空载或固定测试平台上进行验证，合理设置关节限位、速度限制、力矩限制、通信超时和紧急停止等保护措施；未经充分测试和风险评估，不得直接进行人体穿戴实验，不得在道路、楼梯、湿滑地面及其他高风险环境中使用。未成年人或缺乏相关技术能力的人员，应在专业人员指导下开展开发与测试。

因开发者擅自修改软硬件、使用不当、违反安全要求、超出产品设计用途或接入未经验证的第三方设备而造成的损失，由相关责任方在法律允许的范围内承担相应责任。二次开发内容原则上不属于原产品的标准技术支持和质量保证范围，具体以产品保修政策、销售合同及开源许可证为准。本声明不排除或限制依法不能免除的产品质量责任、人身损害责任，以及因故意或重大过失造成损失所应承担的责任。

开发者开始二次开发，即视为已阅读、理解并接受上述风险提示及责任边界。如不同意本声明，请勿修改、编译、烧录或运行相关软硬件。

目录

一、前言.....	6
1.1 前言.....	6
1.2 注意事项.....	6
二、环境安装和代码烧录.....	8
2.1 keil 安装.....	8
2.2 cubemx 安装.....	12
2.3 MobaXterm 安装.....	12
2.4 烧录接口.....	13
三、嵌入式 API 讲解	15
3.1 调度器.....	15
3.1.1 调度器概述.....	15
3.1.2 前台与后台.....	15
3.1.3 后台调度器.....	16
3.2 用户接口.....	17
3.3 电机控制函数.....	18
3.3.1 控制模式与对应接口.....	18
3.3.2 标准控制流程.....	19
3.3.3 控制函数说明.....	19
3.3.4 典型调用示例.....	21
3.3.5 电机控制注意事项.....	22
3.4 具体 API 和变量讲解	22
3.4.1 开发入口与数据读取原则.....	22
3.4.2 公共枚举.....	23
3.4.3 公开数据对象汇总.....	24
3.4.4 IMU 数据结构体与变量.....	24
3.4.5 气压计数据结构体与变量.....	25
3.4.6 电机反馈数据结构体与变量.....	25
3.4.7 电池电量变量.....	26

3.4.8 数据刷新函数.....	26
3.4.9 语音和 LED 接口	27
3.4.10 USART1 用户 JustFloat 发送接口.....	27
3.4.11 统一返回值.....	28
3.4.12 电机关机与主控按键唤醒.....	29
3.4.13 用户开发边界.....	29
3.5 shell 调试.....	30

一、前言

1.1 前言

为帮助开发者进一步了解和拓展驭介智能科教外骨骼的功能，我们编写了本二次开发手册。本手册面向具备嵌入式开发、机器人控制或机电系统基础的高校师生、科研人员及开源爱好者，主要介绍系统架构、硬件资源、通信协议、电机控制模式、软件接口及调试方法，为功能验证、算法研究和应用创新提供参考。

驭介智能坚持开放、协作的理念，希望通过开源软硬件平台降低外骨骼技术的学习与开发门槛。开发者可以在现有系统基础上开展传感器数据采集、步态识别、助力算法、运动控制及人机交互等方向的研究，但二次开发可能改变设备原有的运动特性和安全机制。修改程序或控制参数后，应先在断电、空载或固定测试平台上完成验证，并设置合理的关节限位、速度限制、力矩限制和紧急停止措施；未经充分测试，不得直接进行人体穿戴实验。希望本手册能够帮助开发者安全、规范地完成二次开发，并与我们共同探索开源外骨骼在科教与科研领域的更多可能。

1.2 注意事项

请仔细阅读免责声明，一旦开始二次开发，即视为已阅读、理解并接受上述风险提示。

1. 烧录二次开发程序将覆盖设备内原有固件，覆盖后无法直接恢复。如需恢复原厂固件，请联系驭介智能售后人员处理。
2. 修改关节运动范围、控制方向或位置参数前，应确认机械限位及左右关节方向。错误参数可能造成关节超限、结构碰撞、连接件变形或绑带组件损坏。
3. 禁止设置超过设备允许范围的力矩、速度和位置指令。过大的控制参数、长期堵转、频繁换向或异常振荡可能造成电机、减速器及驱动器过热或永久损坏。
4. 使用 MIT 运控模式时，应从较小的刚度、阻尼和前馈力矩开始逐步调试。参数设置不当可能导致关节剧烈振动、突然运动或产生过大的交互力。

5. 硬件改装及外部设备接入前，应确认供电电压、接口定义、信号电平和接线极性。错误接线、短路或带电插拔可能损坏主控板、电机驱动器、传感器及电池系统。
6. 二次开发造成的固件异常、结构损坏、电机损坏或其他故障，将不属于标准保修范围。发现异响、卡滞、异常发热时，应立即停止运行并切断电源，联系售后人员处理。

二、环境安装和代码烧录

2.1 keil 安装

我们提供了 5.42 版本的 keil 安装包，如果您的版本低于此版本，请升级。过低的版本无法下载 STM32H5 系列的芯片。

- 1.打开驭介智能科教外骨骼-嵌入式资料\3.嵌入式开发软件\mdk542a .exe

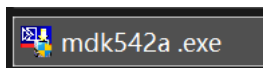


图 2.1 keil 安装包

- 2.如图所示，点击 Next:

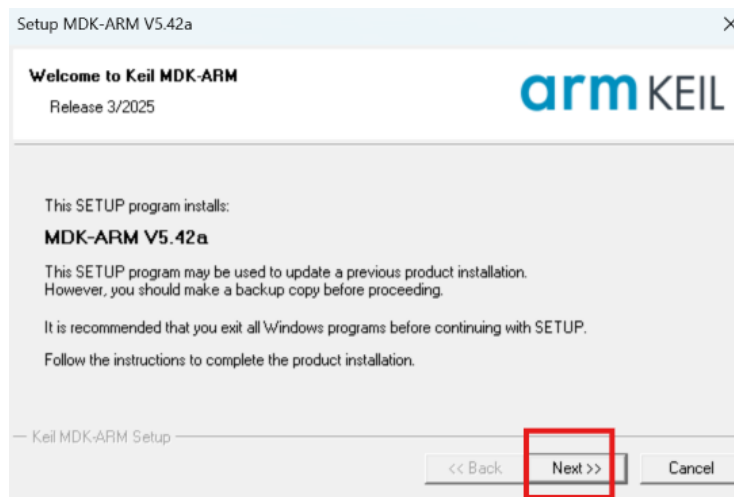


图 2.2 keil 安装过程 1



图 2.3 keil 安装过程 2

3.选择安装的 keil 路径和 pack 路径



图 2.4 keil 安装过程 3

4.下面图片随便填

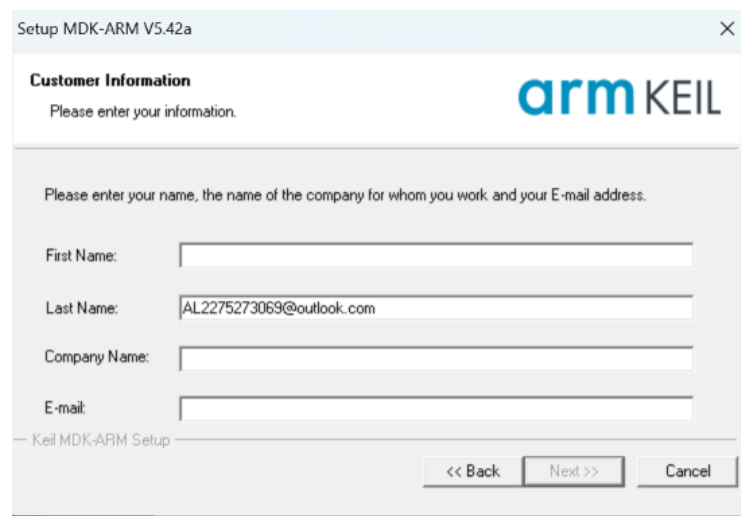


图 2.5 keil 安装过程 4

5.安装完毕后，右键以管理员身份运行 keil 软件。

6.接下来进行 keil 破解，点击如图所示。

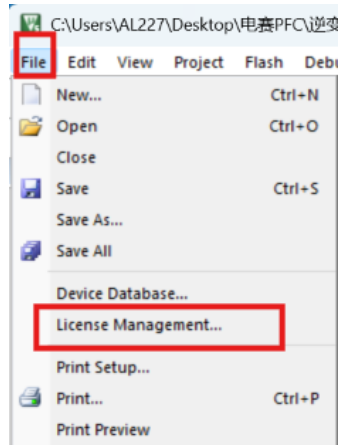


图 2.6 keil 破解过程 1

7.复制 CID

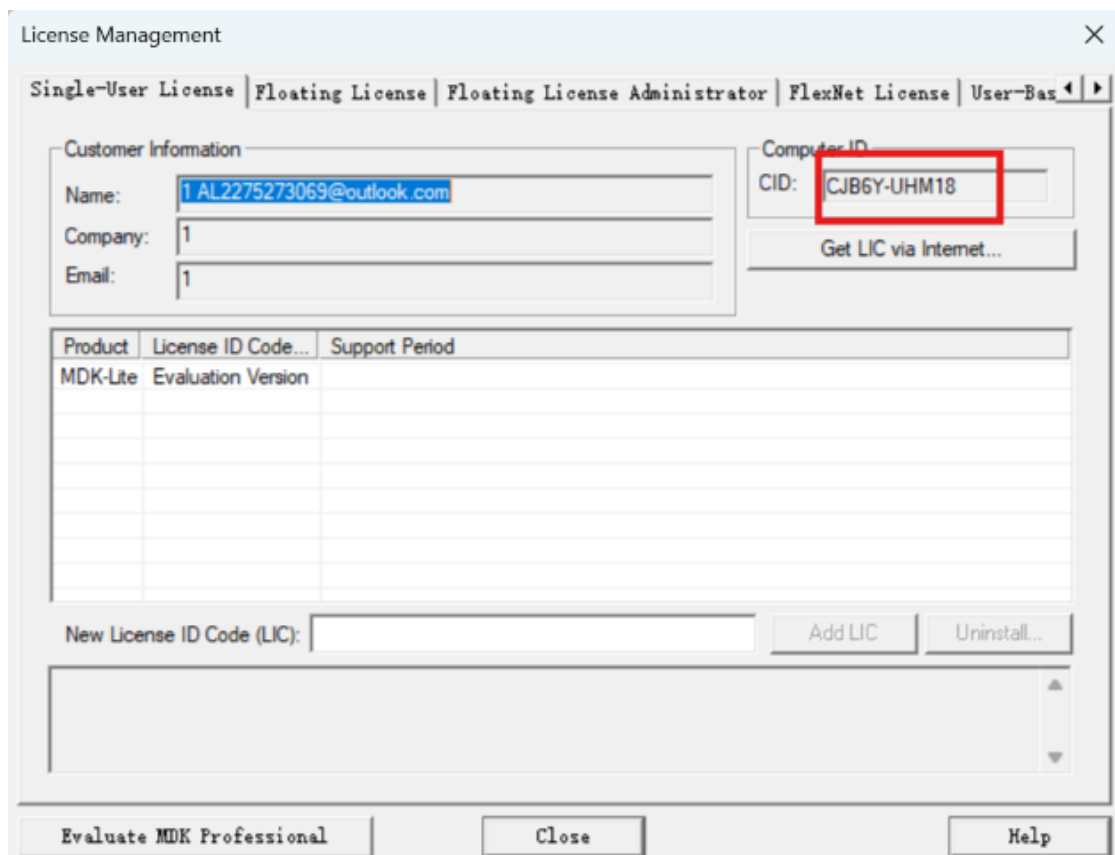


图 2.7 keil 破解过程 2

7. 打开提供的破解软件，注意要关闭你的所有的杀毒软件！

 keygen_new2032.zip

图 2.8 keil 破解过程 3

8.选择 ARM，将上面复制的 CID 粘贴进去，点击 Generate 生成密钥



图 2.9 keil 破解过程 4

9. 将密钥粘贴回来

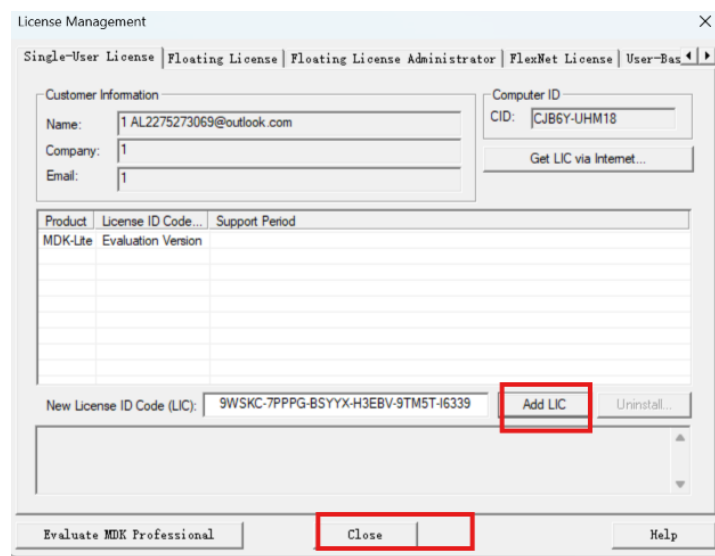


图 2.10 keil 破解过程 5

10.安装 PACK 包，我们提供了 F1，F4，H5 系列，如需要其他系列，可以联系售后人员。

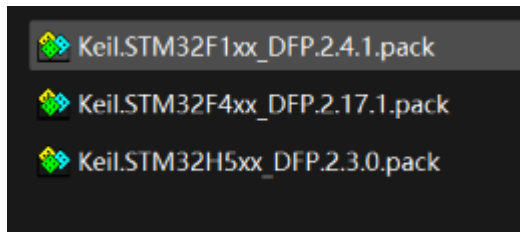


图 2.11 keilpack 包

11.打开我们提供的例程，编译。提示无错误

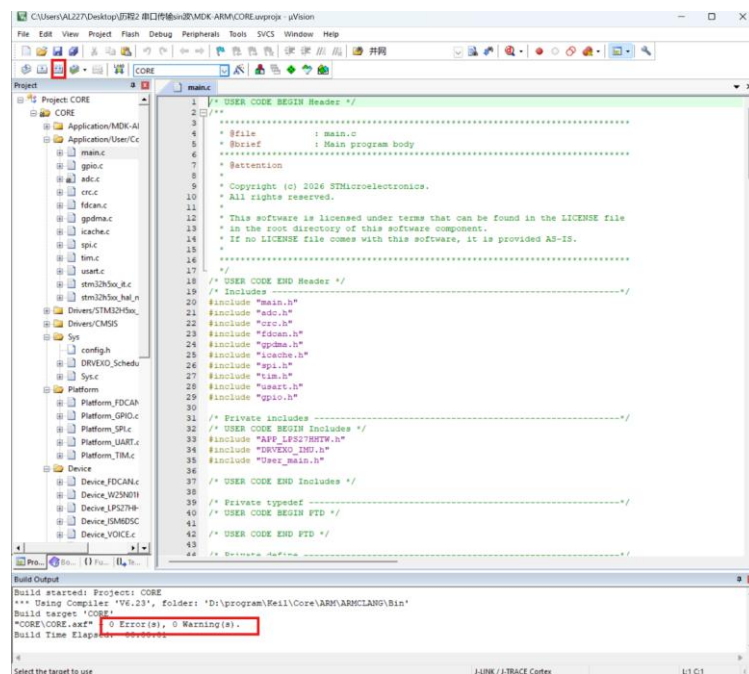


图 2.12 keil 编译验证

2.2 cubemx 安装

打开我们提供的 Cubemx6.17 的安装包，如果您自带的版本过低，请升级。过低的版本无法打开我们的 cubemx 工程。

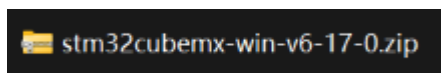


图 2.12 Cubemx 安装包

2.3 MobaXterm 安装

打开驭介智能科教外骨骼-嵌入式资料\3.嵌入式开发软件\MobaXterm，复制出来即可。

2.4 烧录接口

1.打开烧录口上方保护盖，部分机器上打有防水胶，您可以强行破开。



图 2. 烧录口示意图

2.接下来您可以看到如下接口：

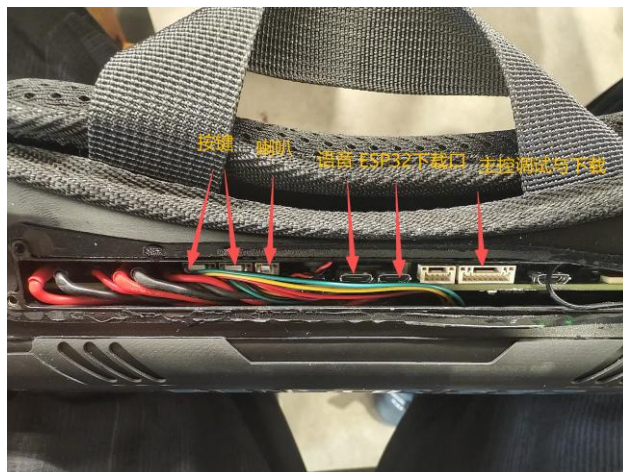


图 2 烧录接口示意图

3.下载器的接口定义如图所示，当然，您可以通过我们的外置转接板快速扩展。

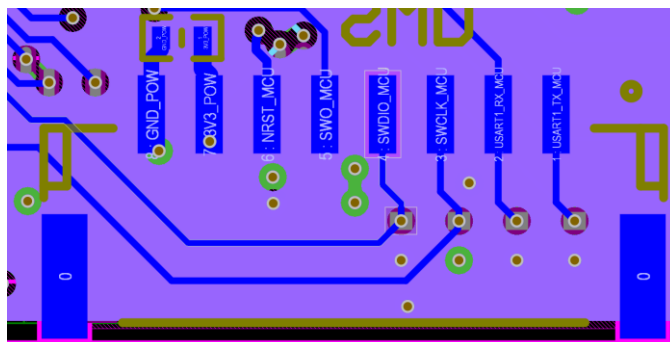


图 2 烧录接口定义示意图

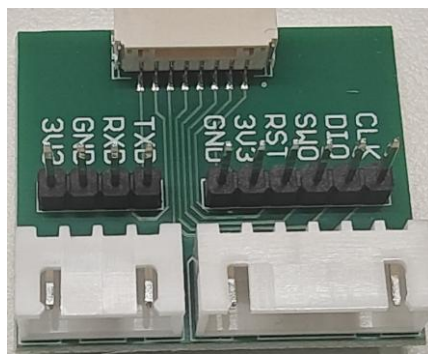


图 2 扩展板示意图

4.按照如下图连线将我们提供的加长板下载器进行连接。**在此特别提醒，请将绿色和紫色转接板的金属裸露部分用胶带或者其他物品包裹，避免短路。**

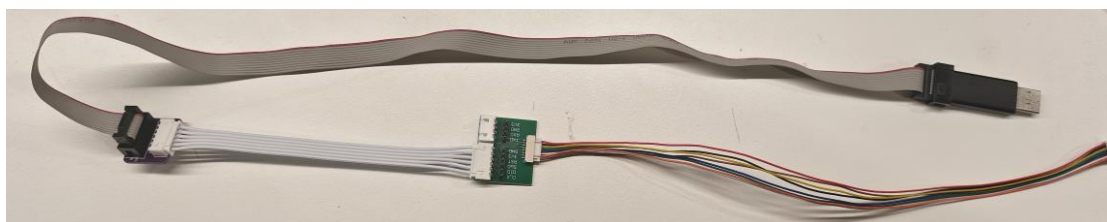


图 2 扩展板连接示意图

5.**第一次下载时需要按住任意电机的一个按键，由于主控处于休眠状态，可能会无法下载代码！**

三、嵌入式 API 讲解

3.1 调度器

3.1.1 调度器概述

本工程采用基于 TIM7 定时器的轻量级后台周期调度器。调度器以 1 ms 为基本时间单位，根据任务周期和优先级依次执行 IMU 数据采集、姿态更新、气压计处理、FDCAN 通信、语音控制、LED 控制及低功耗检测等系统任务。

调度器具有结构简单、资源占用少和任务周期明确等特点，适用于任务数量较少、执行时间较短的嵌入式应用。用户可根据二次开发需求，在现有任务列表中添加自定义周期任务。

3.1.2 前台与后台

前台主循环是 User_main()函数。系统完成硬件和软件初始化后进入该函数，用户可在其中编写二次开发业务逻辑。

```
/**
 * @brief 用户二次开发主函数，系统就绪后进入并持续运行
 * @param 无
 * @return 无
 */
void User_main(void)
{
    while (1)
    {
        /* 在此处编写用户二次开发代码。 */
    }
}
```

图 3.1 用户入口函数

后台任务是指由调度器按照固定周期自动执行的任务。TIM7 每 1 ms 产生一次中断，随后调用 Scheduler_Run()检查任务列表。当某个任务达到设定周期时，调度器便调用对应的任务函数。需要注意的是，本工程的后台任务实际运行在 TIM7 中断上下文中，因此任务函数必须保持简短，不得包含延时、长时间循环或其他阻塞操作。

3.1.3 后台调度器

每个任务均具有独立的运行周期，单位为毫秒。调度器每 1 ms 检查一次任务，当任务距离上次运行的时间达到设定周期后执行该任务。

例如，任务周期设置为 10 ms，表示调度器正常情况下每 10 ms 调用一次该任务。

任务优先级的有效范围为 0~31，数值越小，执行顺序越靠前。调度器最多支持 20 个静态任务。

系统初始化时会自动创建以下周期任务：

```
Scheduler_CreateStaticTask(IMU_Data_Task, "IMUDATA", 10, 3, &IMUDataPollingHandle);
if(Sub_IMU_Flag)
    Scheduler_CreateStaticTask(IMU_Data_Sub_Task, "IMUDATA2", 10, 3, &IMUData2PollingHandle);
Scheduler_CreateStaticTask(IMU_Updata_Task, "IMUUpdata", 10, 4, &IMUUpdataPollingHandle);
Scheduler_CreateStaticTask(LPS27HHTW_Task, "Barometer", 1, 5, &BarometerPollingHandle);
Scheduler_CreateStaticTask(APP_FDCAN_Task, "FDCAN", 1, 21, &FDCANHandle);

Scheduler_CreateStaticTask(APP_Voice_Task, "Voice", 1, 21, &VoioceHandle);
Scheduler_CreateStaticTask(APP_LED_Task, "LED", 1000, 21, &LEDHandle);
if(Scheduler_CreateStaticTask(APP_LowPower_Task, "LowPower", 10, 22, &LowPowerHandle) != pdPASS)
{
    sys_err.LowPower_Task_Create_err_cnt++;
}
```

图 3.2 调度器列表

任务名称	运行周期	优先级	主要功能
IMUDATA	10 ms	3	主IMU数据采集
IMUDATA2	10 ms	3	副IMU数据采集
IMUUpdata	10 ms	4	IMU姿态融合更新
Barometer	1 ms	5	气压计数据处理
FDCAN	1 ms	21	FDCAN通信维护
Voice	1 ms	21	语音播放队列处理
LED	1000 ms	21	LED状态更新
LowPower	10 ms	22	低功耗状态检测

图 3.3 调度器函数说明

用户自定义任务必须使用以下函数格式：

```
void User_Task(uint16_t dT_ms)
{
    /* 用户周期任务代码。 */
}
```

图 3.4 调度器函数格式

调度器提供动态任务和静态任务两种创建方式。

```
BaseType_t Scheduler_CreateTask(TaskFunction_t pxTaskCode,
                                const char *pcName,
                                uint16_t usRateMs,
                                uint8_t ucPriority,
                                TaskHandle_t *pxCreatedTask);

BaseType_t Scheduler_CreateStaticTask(TaskFunction_t pxTaskCode,
                                       const char *pcName,
                                       uint16_t usRateMs,
                                       uint8_t ucPriority,
                                       TaskHandle_t *pxCreatedTask);
```

图 3.5 调度器创建任务函数

参数	说明
pxTaskCode	需要周期执行的任务函数
pcName	任务名称，建议使用不超过15字节的英文名称
usRateMs	任务周期，单位为毫秒，不能为0
ucPriority	任务优先级，取值范围为0~31
pxCreatedTask	用于保存创建成功后的任务句柄

图 3.6 调度器创建任务函数说明

Scheduler_CreateStaticTask(你需要运行的函数,名称，周期时间，优先级，句柄);

3.2 用户接口

用户代码位于 User_Data 分组：

1. User_Data/Inc/User_main.h

声明用户开发入口 void User_main(void)。

2. User_Data/Src/User_main.c

用户主要编辑此文件。系统完成硬件、传感器、电机和后台任务初始化后，会进入 User_main()。User_main() 内部的 while (1) 是用户永久运行区。

3. User_Data/Inc/User_Data.h

提供传感器数据结构体、公开数据变量、语音、LED 和电机控制接口。

平台的 IMU、气压计、FDCAN、电机反馈、语音和 LED 任务由 TIM7 中断调度器在后台运行。用户不需要在 User_main() 中调用平台调度器。

系统启动后台 FDCAN 后会最多等待 1 秒获取有效电池帧；收到数据时先播报当前电池电量，再进入 User_main()。未连接电池时超时时直接进入 User_main()。
重要注意事项：

- 1.不要在用户代码中长时间关闭中断，否则后台传感器和通信任务会停止。
- 2.User_main() 不应返回；用户业务应持续写在其 while(1) 中。
- 3.用户应优先读取 User_Data 提供的变量并调用 User_ 开头的接口，避免直接依赖 APP、Device、Platform 或 Sys 内部模块。
- 4.传感器和电机反馈由中断后台异步刷新。需要组合使用多个字段时，建议先复制用户自己的局部结构体，再进行计算，避免重复读取正在变化的数据。

3.3 电机控制函数

本节介绍左右髋关节电机的控制模式、标准调用流程及公开控制函数。模式设置、使能和失能命令默认等待电机应答 100 ms；目标值设置函数负责发送控制指令，不等待电机完成运动。

3.3.1 控制模式与对应接口

控制模式	模式设置接口	目标值接口	主要说明
力矩模式	User_Motor_Mode_Set()	User_Motor_Torque_Set()	mode 取 TORQUE；直接设置目标力矩，单位及范围以电机固件为准。
速度模式	User_Motor_Mode_Set()	User_Motor_Velocity_Set()	mode 取 VELOCITY；设置目标速度，单位为 rad/s。
位置模式	User_Motor_Mode_Set()	User_Motor_Position_Set()	mode 取 POSITION；设置目标位置，单位为 rad，底层采用二阶位置滤波。
MIT 模式	User_Motor_MIT_Mode_Set()	User_Motor_MIT_Set()	联合设置位置、速度、KP、KD 和前馈力矩。

模式设置函数只切换控制器模式，不会自动使能电机；电机使能后，应持续

按照当前模式发送对应目标值。禁止在模式不一致时发送控制指令。

3.3.2 标准控制流程

确认左右电机已经连接，并已收到有效 FDCAN 反馈；故障码非 0 或反馈异常时不得继续控制。

保持目标电机处于失能状态，调用模式设置函数选择力矩、速度、位置或 MIT 模式。

检查模式设置函数返回值，返回 0 后再调用 `User_Motor_Enable()` 使能电机。

根据当前模式周期发送目标值，并持续监测位置、速度、力矩、电流、温度和故障码。

停止控制、发生通信异常或检测到故障时，立即停止发送目标值并调用 `User_Motor_Disable()`。

3.3.3 控制函数说明

(1) `User_Motor_Mode_Set`

```
int32_t User_Motor_Mode_Set(User_Motor_direction_en direction,
                             User_Motor_control_mode_en mode);
```

功能：将指定电机设置为力矩、速度或位置控制模式，不执行电机使能。位置模式固定映射为电机固件的二阶位置滤波模式。

参数：`direction` 指定左电机或右电机；`mode` 指定力矩、速度或位置模式。

返回值：0 表示设置成功；负值表示参数错误、发送失败或等待响应超时。

说明：建议在电机失能状态下切换模式。

(2) `User_Motor_MIT_Mode_Set`

```
int32_t User_Motor_MIT_Mode_Set(User_Motor_direction_en direction);
```

功能：将指定电机设置为 MIT 联合控制模式，不执行电机使能。

参数：`direction` 只能为 `USER_MOTOR_LEFT` 或 `USER_MOTOR_RIGHT`。

返回值：0 表示设置成功；负值表示参数错误、发送失败或等待响应超时。

(3) `User_Motor_Enable`

```
int32_t User_Motor_Enable(User_Motor_direction_en direction);
```

功能：使能指定电机，使其进入可接收运动控制指令的状态。

参数：`direction` 指定左电机或右电机。

返回值：0 表示使能成功；负值表示参数错误、发送失败或等待响应超时。

说明：调用前必须先完成模式设置，并准备安全的初始目标值。

(4) User_Motor_Disable

```
int32_t User_Motor_Disable(User_Motor_direction_en direction);
```

功能：失能指定电机，停止其主动运动输出。

参数：direction 指定左电机或右电机。

返回值：0 表示失能成功；负值表示参数错误、发送失败或等待响应超时。

(5) User_Motor_Torque_Set

```
int32_t User_Motor_Torque_Set(User_Motor_direction_en direction,  
                               float torque);
```

功能：发送力矩模式的目标力矩。

参数：direction 指定电机；torque 为目标力矩，单位和安全范围由电机通信协议及电机固件定义。

返回值：0 表示发送成功；负值表示参数错误或发送失败。

(6) User_Motor_Velocity_Set

```
int32_t User_Motor_Velocity_Set(User_Motor_direction_en direction,  
                                 float velocity);
```

功能：发送速度模式的目标速度。

参数：direction 指定电机；velocity 为目标速度，单位为 rad/s。

返回值：0 表示发送成功；负值表示参数错误或发送失败。

(7) User_Motor_Position_Set

```
int32_t User_Motor_Position_Set(User_Motor_direction_en direction,  
                                 float position);
```

功能：发送位置模式的目标位置。

参数：direction 指定电机；position 为目标位置，单位为 rad。

返回值：0 表示发送成功；负值表示参数错误或发送失败。

(8) User_Motor_MIT_Set

```
int32_t User_Motor_MIT_Set(User_Motor_direction_en direction,
                           float position,
                           float velocity,
                           float kp,
                           float kd,
                           float torque);
```

功能：发送 MIT 联合控制参数，使电机根据目标位置、目标速度、位置增益、速度增益和前馈力矩计算输出。

参数：direction 指定电机；position 和 velocity 分别为目标位置与速度；kp 和 kd 分别为位置增益与速度增益；torque 为前馈力矩。

返回值：0 表示发送成功；负值表示参数错误或发送失败。

说明：用户层不限制 position、velocity、kp、kd 和 torque 的数值，调用前必须依据电机协议确认安全范围。

3.3.4 典型调用示例

(1) 速度模式示例

```
int32_t ret;
ret = User_Motor_Mode_Set(USER_MOTOR_LEFT,
                          USER_MOTOR_CONTROL_MODE_VELOCITY);
if (ret == 0)
{
    ret = User_Motor_Enable(USER_MOTOR_LEFT);
}
if (ret == 0)
{
    ret = User_Motor_Velocity_Set(USER_MOTOR_LEFT, 1.0f);
}
if (ret < 0)
{
    (void)User_Motor_Disable(USER_MOTOR_LEFT);
}
```

(2) MIT 模式示例

```
int32_t ret;
ret = User_Motor_MIT_Mode_Set(USER_MOTOR_LEFT);
if (ret == 0)
{
    ret = User_Motor_Enable(USER_MOTOR_LEFT);
}
if (ret == 0)
{
    ret = User_Motor_MIT_Set(USER_MOTOR_LEFT,
                             0.0f, 0.0f,
                             0.0f, 0.0f, 0.0f);
}
```

示例说明：MIT 示例中的参数仅用于展示接口调用顺序，不代表实际穿戴参数。调试时应从较小的 KP、KD 和前馈力矩开始，并逐步验证关节响应。

3.3.5 电机控制注意事项

`direction` 只能为 `USER_MOTOR_LEFT` 或 `USER_MOTOR_RIGHT`。

力矩、速度、位置目标函数必须与当前选择的控制模式对应。

每次调用均应检查返回值；返回负值时，不得继续执行依赖该步骤的控制命令。

用户层不限制目标位置、速度、力矩、KP 和 KD，开发者必须依据电机协议设置安全范围。

电机故障码非 0、反馈异常、通信未准备好或温度异常时，应停止输出运动指令并失能电机。

首次验证应采用空载或固定平台，禁止直接以人体穿戴方式验证未经测试的控制参数。

3.4 具体 API 和变量讲解

本节对 `User_Data` 模块面向二次开发者公开的枚举、数据结构体、全局变量和功能接口进行统一说明。用户代码应优先读取 `User_Data` 提供的数据，并调用 `User_` 前缀接口，避免直接依赖 APP、Device、Platform 或 Sys 内部实现。

3.4.1 开发入口与数据读取原则

用户二次开发入口为 `User_Data/Src/User_main.c` 中的 `User_main()` 函数。系统完成硬件、传感器、FDCAN 和后台任务初始化后进入该函数，`User_main()` 内部

的 while (1)为用户业务永久运行区，函数不应返回。

```
void User_main(void)
{
    while (1)
    {
        /* 在此处编写用户二次开发代码。 */
    }
}
```

不要长时间关闭中断，否则后台传感器采集、FDCAN 通信和周期任务将停止。

传感器与电机反馈由后台异步刷新；组合使用多个字段时，建议先复制到用户局部变量或私有结构体，再进行计算。

主动刷新函数不会等待新数据，只会复制平台当时已有的最新数据。

3.4.2 公共枚举

(1) 电机方向枚举 User_Motor_direction_en

枚举值	数值	说明
USER_MOTOR_LEFT	0	左电机。
USER_MOTOR_RIGHT	1	右电机。

(2) 电机控制模式枚举 User_Motor_control_mode_en

枚举值	数值	说明
USER_MOTOR_CONTROL_MODE_TORQUE	0	力矩模式。
USER_MOTOR_CONTROL_MODE_VELOCITY	1	速度模式。
USER_MOTOR_CONTROL_MODE_POSITION	2	位置模式，底层采用二阶位置滤波。

(3) 电机系统模式枚举 User_Motor_system_mode_en

枚举值	数值	说明
USER_MOTOR_SYSTEM_MODE_UNKNOWN	0	未知模式或尚未收到有效反馈。
USER_MOTOR_SYSTEM_MODE_REST	1	休息模式，对应电机原始值 5。
USER_MOTOR_SYSTEM_MODE_COMFORT	2	舒适模式，对应电机原始值 0。
USER_MOTOR_SYSTEM_MODE_SPORTS	3	运动模式，对应电机原始值 2。

枚举值	数值	说明
USER_MOTOR_SYSTEM_MODE_DAMPING	4	阻尼模式，对应电机原始值 4。

注意： system_mode 表示电机周期反馈中的系统运行模式，不是 User_Motor_MIT_Mode_Set() 设置的电机控制器模式。

(4) 电机电矩挡位枚举 User_Motor_torque_level_en

枚举值	数值	原始反馈值
USER_MOTOR_TORQUE_LEVEL_UNKNOWN	0	未定义或尚未收到有效反馈
USER_MOTOR_TORQUE_LEVEL_1	1	25
USER_MOTOR_TORQUE_LEVEL_2	2	50
USER_MOTOR_TORQUE_LEVEL_3	3	75
USER_MOTOR_TORQUE_LEVEL_4	4	100

3.4.3 公开数据对象汇总

公开变量	数据类型	更新来源	主要内容
User_IMU_Data	User_IMU_data_t	IMU 姿态任务	角速度、加速度、欧拉角、四元数
User_Barometer_Data	User_Barometer_data_t	气压计任务	气压、海拔、温度、有效标志
User_Motor_Data	User_Motor_data_t	FDCAN 任务	左右电机状态及完整运动反馈
User_Battery_Level	uint8_t	FDCAN 电池任务	电池电量百分比

上述变量均由平台后台任务自动更新，用户通常直接读取即可；如需立即复制平台当前快照，可调用对应的 Update 函数。

3.4.4 IMU 数据结构体与变量

```
extern struct User_IMU_data_t User_IMU_Data;
```

字段	单位	说明
user_gyro.x / y / z	dps	三轴角速度。
user_acc.x / y / z	g	三轴加速度。

字段	单位	说明
user_euler.roll	°	横滚角。
user_euler.pitch	°	俯仰角。
user_euler.yaw	°	航向角。
user_quaternion.w	无	四元数实部。
user_quaternion.x / y / z	无	四元数虚部。

```
User_IMU_Data_Update();
float roll = User_IMU_Data.user_euler.roll;
```

3.4.5 气压计数据结构体与变量

```
extern struct User_Barometer_data_t User_Barometer_Data;
```

字段	单位	说明
pressure_hpa	hPa	当前气压。
altitude_m	m	当前海拔。
temperature_deg_c	℃	气压计温度。
altitude_ready	无	海拔有效标志；为 1 时方可使用 altitude_m 参与控制。

```
User_Barometer_Data_Update();
if (User_Barometer_Data.altitude_ready == 1U)
{
    float altitude = User_Barometer_Data.altitude_m;
    (void)altitude;
}
```

3.4.6 电机反馈数据结构体与变量

```
extern struct User_Motor_data_t User_Motor_Data;
```

字段	说明
fault_code	电机故障码；0 通常表示当前未报告故障，其他值以电机协议为准。
power_status	电机系统电源状态原始值，取自反馈状态字节高 4 位。
system_mode	当前系统运行模式，类型为 User_Motor_system_mode_en。

字段	说明
torque_level	当前力矩挡位，类型为 User_Motor_torque_level_en。
input_position / output_position	电机输入端位置与输出端位置。
torque / current	电机实际反馈力矩与电流。
input_velocity / output_velocity	电机输入端速度与输出端速度。
input_acceleration / output_acceleration	电机输入端加速度与输出端加速度。
temperature_deg_c	电机温度，单位为摄氏度。

User_Motor_Data.left 和 User_Motor_Data.right 分别保存左右电机的完整反馈。位置、速度、加速度、力矩和电流保持电机通信协议的物理单位，用户层不进行二次缩放。

```
User_Motor_Data_Update();
float left_torque = User_Motor_Data.left.torque;
float right_temp = User_Motor_Data.right.temperature_deg_c;
```

可选外设：左右电机未连接时，系统初始化不会等待电机反馈，也不会阻塞进入 User_main()。发送控制命令前应确认电机已连接，并检查控制函数返回值。

3.4.7 电池电量变量

```
extern volatile uint8_t User_Battery_Level;
```

User_Battery_Level 保存当前电池电量百分比，正常范围为 0~100，单位为%。FDCAN 电池任务在收到有效电池报文后自动刷新该变量；尚未收到有效报文时，变量保持平台当前已有值。

```
User_Battery_Level_Update();
uint8_t battery_level = User_Battery_Level;
```

3.4.8 数据刷新函数

函数	对应变量的名称	功能
User_IMU_Data_Update()	User_IMU_Data	复制当前角速度、加速度、欧拉角和四元数。
User_Barometer_Data_Update()	User_Barometer_Data	复制当前气压、海拔、温度和有效标志。
User_Motor_Data_Update()	User_Motor_Data	复制左右电机当前完整反馈。

函数	对应变量	功能
User_Battery_Level_Update()	User_Battery_Level	复制当前电池电量。

刷新机制：主动刷新不会等待新传感器数据或新 FDCAN 报文，只会复制平台当时已有的最新数据；气压计海拔仍需通过 altitude_ready 判断有效性。

3.4.9 语音和 LED 接口

接口	参数	功能与返回值
User_Voice_Enqueue()	voice_id	将语音编号加入后台播放队列；0 成功，-2 编号无效，-4 队列已满。语音见 8. 其他工具
User_LED_On()	无	停止闪烁并常亮；0 成功，负值表示 GPIO 控制失败。
User_LED_Off()	无	停止闪烁并常灭；0 成功，负值表示 GPIO 控制失败。
User_LED_Blink()	无	进入闪烁模式；0 成功，-7 表示 LED 任务未就绪。
User_LED_Blink_Time_Set()	blink_time_ms	设置每次亮灭翻转间隔；0 成功，-2 参数为 0，-7 任务未就绪。

当闪烁间隔设置为 500 ms 时，LED 每 500 ms 翻转一次状态，一个完整亮灭周期约为 1000 ms。

```
User_LED_On();
User_LED_Off();
User_LED_Blink_Time_Set(500U);
User_LED_Blink();
```

3.4.10 USART1 用户 JustFloat 发送接口

```
int32_t User_UART_Send(const float *channel_data, uint16_t channel_count);
```

项目	配置或说明
发送引脚	PA9（USART1_TX）
串口参数	230400 bit/s，8 位数据位，1 位停止位，无校验位
电平标准	3.3 V TTL；连接主机串口模块时必须共地
发送方式	DMA 非阻塞发送

项目	配置或说明
channel_data	待发送的 float 通道数组，数组顺序对应 VOFA+通道顺序
channel_count	通道数量，范围 1~255
协议帧尾	接口自动添加 00 00 80 7F

接口返回 0 表示 DMA 发送启动成功；负值表示参数无效、上一次 DMA 发送尚未完成或底层发送失败。接口使用内部静态发送帧，必须等待上一帧 DMA 发送完成后再发送下一帧。

```
float send_data[3] = {1.0f, 2.0f, 3.0f};
while (1)
{
    if (User_UART_Send(send_data, 3U) == 0)
    {
        HAL_Delay(100U);
    }
}
```

3.4.11 统一返回值

返回值	名称	说明
0	LY_OK	操作成功。
-1	LY_ERR	通用错误。
-2	LY_ERR_INVALID	参数错误。
-3	LY_ERR_TIMEOUT	等待响应超时。
-4	LY_ERR_BUSY	设备或资源忙。
-5	LY_NO_DATA	当前没有有效数据。
-6	LY_ERR_IO	底层通信或 I/O 失败。
-7	LY_ERR_NOT_READY	设备或后台任务尚未准备好。
-8	LY_ERR_UNSUPPORTED	当前功能不支持。

不同函数只返回与自身业务相关的错误值。用户代码应至少检查返回值是否小于 0，并在错误发生后停止执行依赖当前步骤的后续操作。

3.4.12 电机关机与主控按键唤醒

左右电机均至少收到一帧有效 FDCAN 周期数据后，只要任意一侧反馈的 power_status 为 0，平台会等待当前按键完全释放，然后自动进入 STM32 Standby 低功耗模式。未连接电机或左右电机尚未全部收到有效数据时，不会因缓存初始值为 0 而误进入 Standby。

Standby 会使 CPU、RAM 和大部分外设停止运行，User_main()、TIM7 调度及 FDCAN 后台任务均停止。唤醒后程序从复位入口重新执行，RAM 中未持久化的临时数据不会保留。

唤醒信号	引脚	PWR 唤醒源
RIGHT_WKUP1	PA0	PWR_WKUP1
RIGHT_WKUP2	PA2	PWR_WKUP2
LEFT_WKUP1	PC13	PWR_WKUP4
LEFT_WKUP2	PC1	PWR_WKUP6

按键唤醒后，需要保持任意 WKUP 引脚为高电平满 1 s；达到持续时间后系统重新执行完整初始化并进入 User_main()，提前释放则重新进入 Standby。进入 Standby 前，平台会关闭 VM、5 V 和 3.3 V 外设电源。

```
#define USER_POWER_ON_STANDBY_ENABLE 0U
```

设置为 0：普通上电后直接完成初始化并进入 User_main()，待电机关机状态触发后再进入 Standby。

设置为 1：普通上电时直接进入 Standby；按键唤醒并保持满 1 s 后，系统正常初始化并进入 User_main()。

3.4.13 用户开发边界

User_Data 中的公开变量用于向用户提供传感器、电机和电池数据。
User_前缀函数用于提供数据刷新、语音、LED、串口和电机控制能力。
APP、Device、Platform 和 Sys 属于平台内部实现，后续版本可能调整，不作为稳定的二次开发接口。
用户业务代码应集中放在 User_main.c 中；新增私有结构体和函数时，优先放入 User_Data 目录，避免修改平台后台任务。

3.5 shell 调试

我们提供了一个嵌入式 shell 工具，您可以像 Linux 中一样通过输入命令来进行调试。这可以让您加速开发。（该 shell 和 vofa+无法同时使用）

首先下载我们提供的例程“驭介智能科教外骨骼-嵌入式资料\2.例程代码\历程 11 shell 示例”。我们提供了三个例子，第一个是命令 `ls`，可以查看当前调度器的参数，第二个是 `mode x` 可以切换电机的灯光，第三个是 `set a 1.0f` 可以给变量赋值。

打开“驭介智能科教外骨骼-嵌入式资料\3.嵌入式开发软件\MobaXterm”中的软件。然后连接无线串口。**无线串口我们暂未提供，如需购买可以联系淘宝客服。**

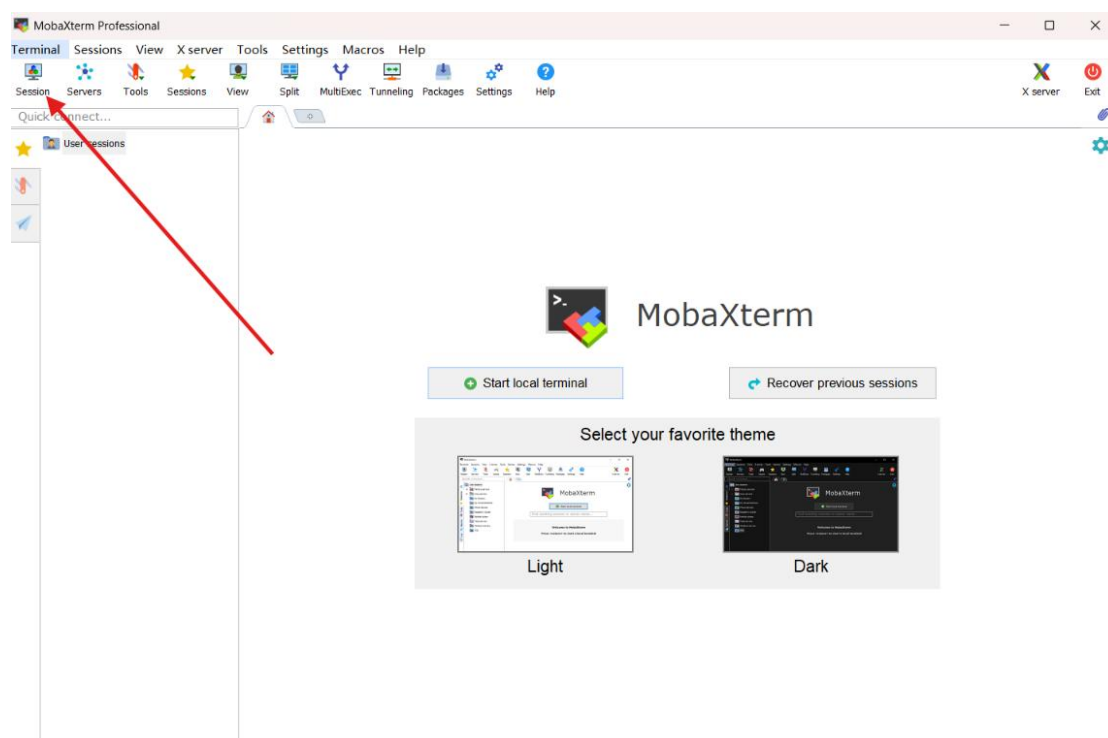


图 3.7 MobaXterm 主界面

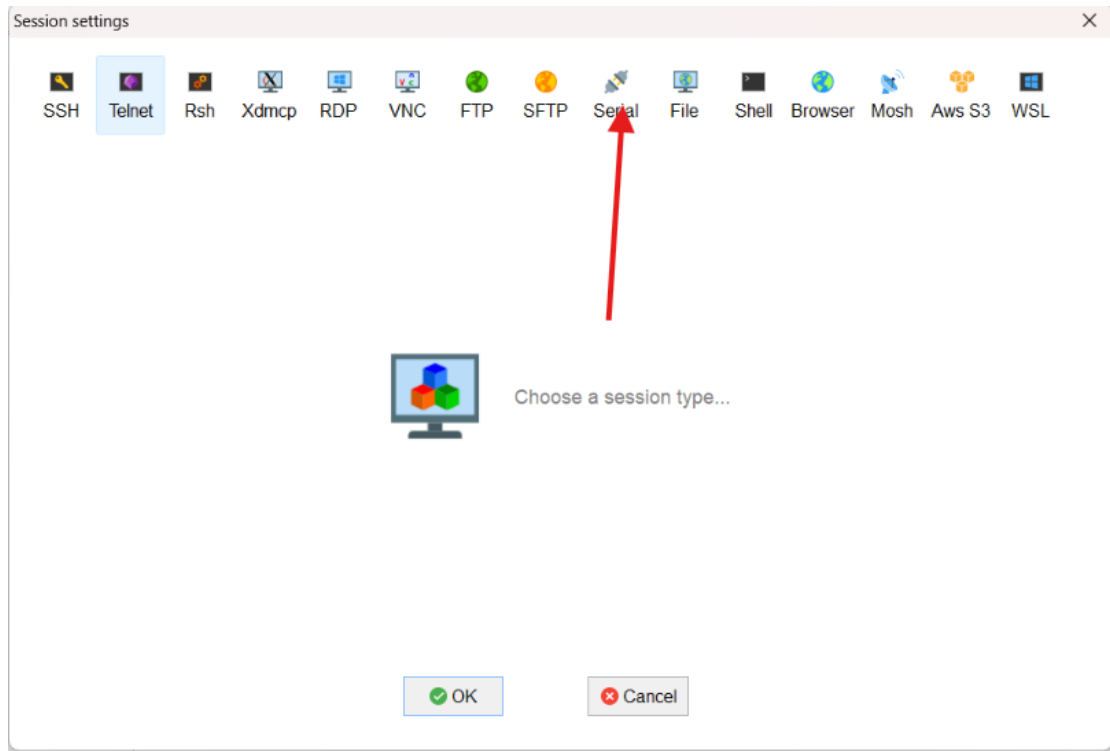


图 3.8 MobaXterm 主界面 2

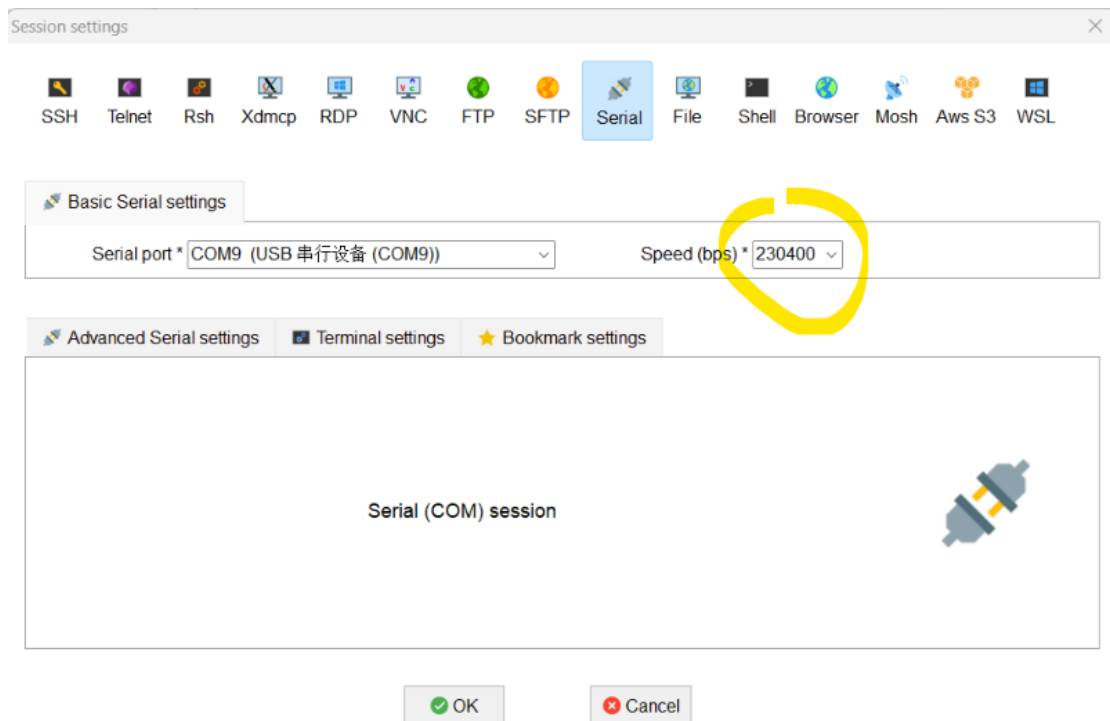


图 3.9 MobaXterm 串口选择

然后给机器上电，可以看到如下界面：

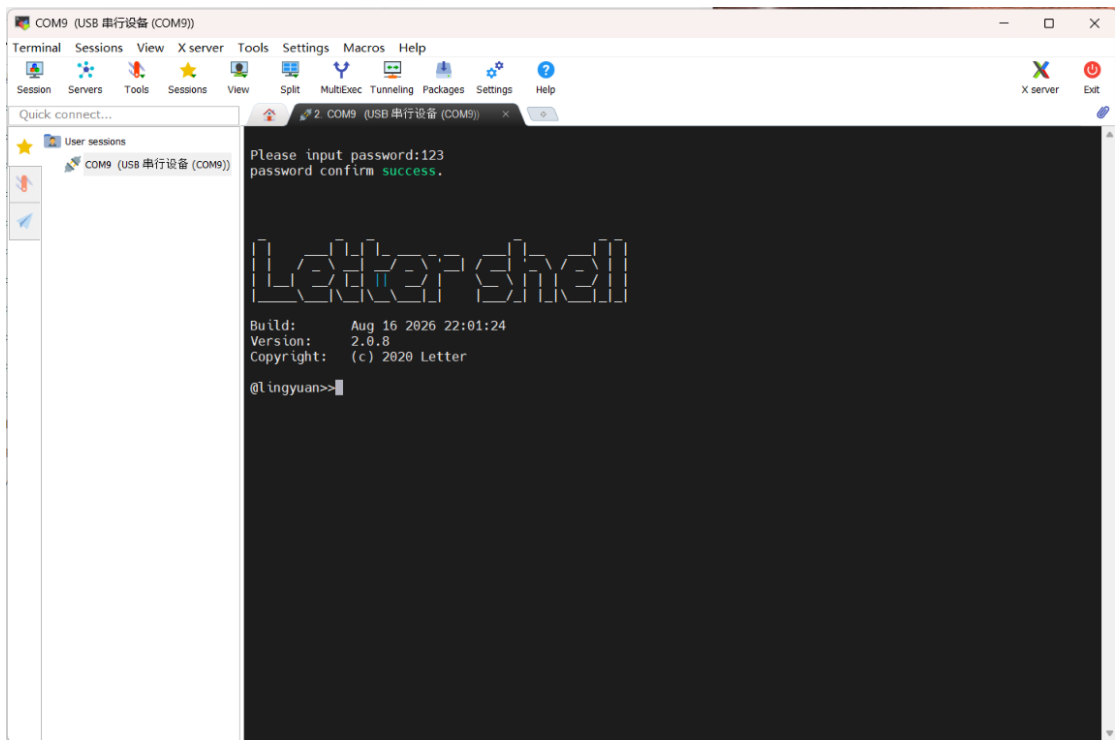


图 3.10 Letter shell 主界面

如下是命令 ls 的示例。

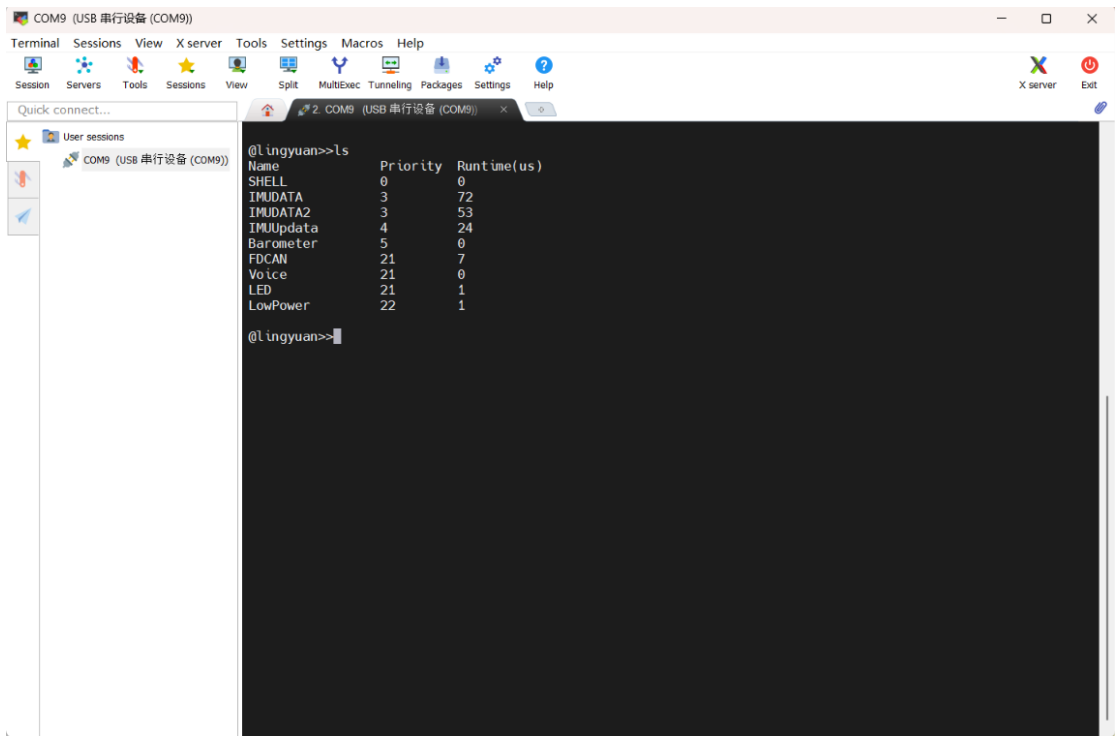


图 3.11 ls 命令

如下是命令 mode 的示例。

```
@lingyuan>>mode 1  
Mode switched to comfort
```

图 3.12 mode 命令

如下是命令 `set` 的示例

```
@lingyuan>>set a 1.0  
a = 1.000000  
  
@lingyuan>>|
```

图 3.13 set 命令

您可以通过输入 `help` 来查看目前的所有指令。

```
func          --test  
reboot        --reboot the mcu  
plus          --plus  
ls            --View scheduler list  
mode          --switch system mode  
set           --set float variable  
setVar        --set var  
vars          --show vars  
help          --command help  
cls           --clear command line
```

图 3.14 help 命令

您可以在代码中如下位置去配置 `shell` 的设置。自定义命令集写在 `cmd.c` 中即可。如果您想在其他代码中使用此 `shell` 脚本，您可以在我们的知识星球寻求帮助或者在 `b` 站上搜索相关教程。

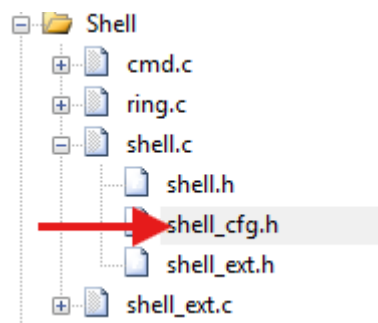


图 3.15 shell 代码配置